Journal of Ambient Intelligence and Humanized Computing
https://doi.org/10.1007/s12652-022-04196-y

ORIGINAL RESEARCH

A fully flexible key delegation mechanism with efficient fine-grained access control in CP-ABE

Shardha Porwal[1] · **Sangeeta Mittal**[1]

Received: 19 April 2021 / Accepted: 15 June 2022
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2022

Abstract
Ciphertext Policy-Attribute Based Encryption (CP-ABE) is a secure one-to-many asymmetric encryption schemes where access control of a shared resource is defined in terms of a set of attributes possessed by a user. Key delegation service, where a user is able to temporally transfer its full or partial access rights, is an important requirement to make CP-ABE based systems practically useful. In existing CP-ABE delegation schemes, a delegator can delegate its entire or partial set of attributes such that all content accessible using those attributes becomes available to the delegatee also. Flexibility to delegate some specific content, which is a practical requirement, is not feasible with these schemes. Further, for a delegated resource, flexibility of fine grained delegation like delegating only a specific subset of read, write and update permissions is not yet available. In this work, CP-ABE key delegation has been enhanced to support specific content and specific permission delegation. The proposed scheme is fully flexible as it additionally supports limits on the number of times delegation can be done, delegatee revocation and traceability of forbidden delegation. Theoretical and practical analysis show that the proposed scheme achieves its objectives and is computationally efficient as well as secure against chosen plaintext and key abuse attacks.

Keywords Key delegation · Specific-access CP-ABE · Specific content delegation · Specific-access right delegation · Controlled key delegation · Limited access control

1 Introduction

In collaborative work environments, data resources are often shared over a central local or cloud-based server such that all users who possess a certain set of attributes are able to access them. Authenticated access of data implemented by traditional encryption techniques is one-to-one and thus not efficient for many-to-one encryption required in this case. Attribute Based Encryption (ABE) is a cryptographic paradigm that supports one-to-many encryption (Sahai and Waters 2005). Using ABE, a data resource can be encrypted in such a way that it can be decrypted by all those users who possess a set of attributes included in an access policy, specified by the owner of the resource. Ciphertext Policy-Attribute Based Encryption (CP-ABE) is a variant of ABE, where access policy is specified at the time of encryption and embedded in cipher text (Porwal and Mittal 2017). Implementation of CP-ABE involves the role of a trusted central authority that generates and distributes attribute-wise public parameters and secret keys for all authorized users of the system. The data resource owner encrypts the resource using public parameters and an access policy. Only those users whose attribute sets satisfy the access policy, are able to compute the decryption key from their secret key (Bethencourt et al. 2007).

In CP-ABE, key delegation is a practical requirement. In key delegation, a user termed as delegator is able to provisionally transfer its partial or full attributes to another user of the system termed as delegatee. Existing delegation schemes allow delegation of a full set of data resources of the delegator and that too with all rights on each resource (Ibraimi et al. 2009; Jiang et al. 2016). On the other hand, the right to a resource can be more fine-grained like read, write/append and update permissions. With read permission,

✉ Shardha Porwal
shardha.porwal@jiit.ac.in

Sangeeta Mittal
sangeeta.mittal@jiit.ac.in

[1] Department of Computer Science and Engineering and Information Technology, Jaypee Institute of Information Technology, Noida, Uttar Pradesh, India

Published online: 11 August 2022

a delegatee can only read the content and with write permission, a delegatee is able to only append the data while with update permission, a delegatee can read the data, write new data at the end and update previously written data in an existing resource. Flexible key delegation mechanism is required in many real-life applications, for example, a doctor may delegate prescription detail of a patient 'xyz' in write/append mode to his/her junior doctor and in update mode to a senior doctor and in read mode to nurse. The doctor may be attending to many patients, key delegation process or existing schemes will provide all the prescriptions to delegatee, and however, there is a need for specific content delegation to delegate the prescription details of only 'xyz'.

In this work, CP-ABE based fully flexible key delegation mechanism has been developed with an objective to implement following delegation capabilities for a delegator:

1. **Specific attribute delegation**: A delegator is able to delegate only a specific set of attributes to the delegatee instead of a full set.
2. **Specific content delegation**: A delegator should be able to delegate specific resources only as compared to no or all resource delegation in existing schemes.
3. **Specific fine-grained access right delegation**: A delegator should be able to specify differential access rights related to a specific resource that he/she wants to delegate to his/her associate.
4. **Specify limit of accessing the delegated content**: A delegator should be able to specify that a delegatee can access the delegated resources only the specified number of times.
5. **Specify limit in further delegation**: A delegator should be able to specify the number of times further delegation can be done.

Figure 1 shows the use case of the proposed delegation scheme. If a doctor is not available then he/she may delegate patient's prescription for read access to nurse by providing k_{read} of the prescription file. The doctor may provide k_{write} to another doctor to add other medicine but cannot modify his prescription and may provide k_{update} to a senior doctor to treat the critical condition of the patient. There is a requirement to extend existing CP-ABE key delegation to implement such low-level access control over resources for the delegated user and make the key delegation process further robust and flexible.

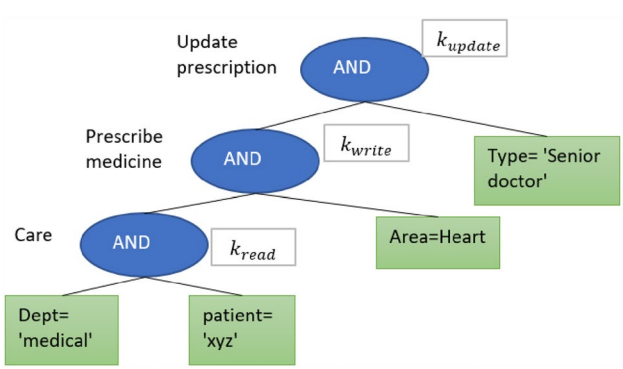


Fig. 1 Use case of proposed fully flexible key delegation

The delegation scheme proposed in this work is termed fully flexible due to the inclusion of all the features given above as 1–5. These have not been implemented by any existing scheme so far. The CP-ABE scheme given in Bethencourt et al. (2007) supports specific attribute delegation. However, specific content delegation, specific rights and other desired features are not present in their scheme. Another work presented in Ibraimi et al. (2009), proposed a mediator based secure key delegation mechanism in CP-ABE. It implements a limit on the number of times re-delegations can be done but doesn't fulfill all other requirements of fully flexible delegation. A traceable CP-ABE scheme given in Jiang et al. (2016) implements a mechanism to capture unauthorized key delegation but lacks other features. The proposed scheme supports fully flexible key delegation and doesn't incur much overhead and is more flexible than available CP-ABE key delegation methods. It has been proved that the proposed scheme is secure against Chosen Plaintext Attack (CPA) under Decisional Diffie Hellman Assumption. Comparative analysis of this work with basic delegation in Bethencourt et al. (2007), flexible further delegation in Porwal and Mittal (2019a) and traceable delegation (Chen et al. 2020) schemes establish that the scheme significantly advances the current state-of-art in this field.

1.1 Research contributions

In this work, we make several research contributions to enhance key delegation in CP-ABE. These contributions are listed as below:

To the best of our knowledge, the proposed work is the first one to design a fine-grained key delegation mechanism in two dimensions.
First Dimension—As per the state-of-art delegation schemes in CP-ABE, contents can only be delegated with full access rights and partial access delegation was not feasible, proposed work implements flexibility to delegate content in any of the read, write and update modes.
Second Dimension—Unlike existing works that implement all-or-nothing approach for delegating content, in the proposed scheme, delegation of specific content rather than delegation of all resources accessible with a specific set of attributes becomes feasible.
Proposed mechanism also supports limited times access to delegated content by delegatee.
Support traceability of delegation use and abuse.

Enhancement in functionality doesn't lead to increase in encryption and decryption cost.

The paper is organized in seven sections. The related work is given in Sect. 2. In Sect. 3, mathematical preliminaries used in the construction of the proposed scheme are described. System model and main functions of the proposed fully flexible key delegation CP-ABE scheme are explained in Sect. 4. The detailed construction of the proposed scheme is presented in Sect. 5. Security, theoretical complexities and the performance analysis of the implemented proposed scheme is elaborated in Sect. 6. The paper is concluded in Sect. 7.

2 Related work

The objective of current work is to provide an efficient fine-grained access control CP-ABE scheme with fully flexible key delegation mechanism. This section provides a summary of the work done in the area of public key cryptography and related security techniques to achieve this. Attribute Based Encryption (ABE), first proposed by Sahai and Waters (2005), is a type of public key cryptography where the secret key of the user is generated by central authority from the set of attributes of the user. Attribute based encryption technique can be implemented using Ciphertext Policy-Attribute Based Encryption (CP-ABE) and Key Policy-Attribute Based Encryption (KP-ABE) (Porwal and Mittal 2017). In CP-ABE techniques, access policy is specified at the time of encryption while in KP-ABE it is specified at the time of key generation.

Key delegation mechanism was first proposed by Sahai and Waters (2005) with a basic ABE scheme, where a delegator can delegate a subset of his/her attributes to a delegatee. Key delegation procedure was also introduced in the seminal paper by Bethencourt et al. (2007) on CP-ABE. Main drawback of their delegation mechanism is that once a delegator delegates his/her attributes, there is no way to prevent further delegation by delegatee.

Flexible further delegation scheme given in Ibraimi et al. (2009) proposed key delegation using a trusted mediator. In this scheme, a delegator can decide whether to allow a delegatee to delegate further by maintaining a delegation list DL at the mediator. Authors in Jiang et al. (2016) proposed a CP-ABE scheme that supports protection against the issue of key delegation abuse and collusion. This scheme provides a traceable CP-ABE mechanism, where the data user who reveals his/her key, can be traced. CP-ABE schemes that support protection against the abuse of key delegation in fog computing have been discussed in Jiang et al. (2016) and Jiang et al. (2018). The scheme presented in Porwal and Mittal (2019a) provides flexibility to delegate for allowing or disallowing further delegation by implementing checks on delegation. Authors in Porwal and Mittal (2019b) proposed a secure key delegation mechanism for CP-ABE with a hidden access structure. In this scheme, a trusted mediator maintains a delegation list DL and a delegator can decide whether to allow a delegatee to delegate further or not.

Xu et al. (2019) presented an ABE scheme that supports randomized piecewise key generation and ciphertext delegation. Ma et al. (2019) presented an efficient CP-ABE scheme that provides online/offline encryption operation and user revocation. Liu et al. (2019) proposed a CP-ABE scheme such that the secret key and malicious traitor can be traced and revoked by the system. Lian et al. (2020) presented a large universe CP-ABE scheme that supports the attribute level user revocation. Cao and Markowitch (2020) proposed circuit CP-ABE based encryption that provides outsourcing of partial decryption. Ali et al. (2020a) presented a fully distributed CP-ABE scheme that supports key delegation and user revocation phases. In this key delegation mechanism, every attribute is supported by several servers hence the user has various options for obtaining attribute based secret keys. Chen et al. (2020) proposed a CP-HABE scheme against key delegation abuse by using a directed graph for an IoT connected healthcare system. Cui et al. (2020) presented a key generation free ABE scheme, where a user can delegate the decryption key and also supports tracing of an abused key to its actual key owner. Ali et al. (2020b) proposed an ABE scheme, where key delegation is managed by several authorities in a distributed way. Authors in Zhang et al. (2021) proposed a CP-ABE scheme with partially hidden access policy. They provided a highly efficient key revocation mechanism for cloud-based systems.

The scheme given in Huang et al. (2018) supports outsourced encryption/decryption to delegate tedious operations to cloud servers and gateway. Authors in Ahuja and Mohanty (2017) provide flexibility to users to delegate their access rights as many-to-one and many-to-many key delegations and group key delegation. Another mechanism is provided in Wang et al. (2017), where a receiver delegates attributes to a cloud server to do an equivalence testing between two data associated with different access policies. Authors in Pussewalage et al. (2017) provided a key delegation functionality that supports controlled key delegation based on the time limit. Authors in Xu et al. (2015) proposed a key delegation mechanism in CP-ABE that supports limited further key delegation based on counting. In this scheme, a token is generated at the time of each delegation that contains the information of all previous delegations made by the data user. The token is generated by authority, thus in this scheme authority needs to be involved in every delegation. Moreover, the size of the token also increases with each delegation.

Authors in Premkamal et al. (2019) proposed a CP-ABE scheme that reduces the computational overhead of

encryption/decryption operations. In this scheme, heavy computations are outsourced to the proxy server and the scheme supports limited data access for users. CP-ABE scheme given in Premkamal et al. (2021) supports dynamic traceability and user revocation in outsourcing big data to the cloud. Their scheme dynamically traces malicious users who share the secret key and automatically revokes such users. Another way of key delegation in CP-ABE is proxy re-encryption. Liang et al. (2013) proposed a CP-ABE scheme with proxy re-encryption that transforms a ciphertext associated with an access policy to another ciphertext with the same plaintext associated with another access policy using a semi-trusted proxy and proxy cannot learn anything about the plaintext. An anonymous CP-ABE scheme with proxy re-encryption termed as "match-then-re-encrypt" has been proposed in Zhang et al. (2016). An additional matching phase is required to be executed before the re-encryption phase to determine that the proxy can perform re-encryption or not.

In all proxy re-encryption-based delegations, an extra encryption step is needed for every delegated ciphertext which reduces the performance of the CP-ABE scheme.

However, the objectives set in this paper haven't been addressed by any of the schemes available in the literature. An overview of availability of proposed delegation functionalities in existing work is given in Table 1. In this work, a fully flexible key delegation mechanism with efficient fine-grained access control supporting specific attribute delegation, specific content delegation, delegation of specific content with specific access mode and limit has been proposed.

Table 1 CP-ABE techniques with key delegation and revocation functionalities

CP-ABE Scheme	Key delegation of partial or full attributes	Controlled Key Delegation	Entities involved in delegation	Delegation key size increase with each further delegation	Attribute revocation	User revocation	Traceability
Sahai and Waters (2005), Bethencourt et al. (2007)	Y	N	DU	N	N	N	N
Ibraimi et al. (2009)	Y	Y	DU	N	Y	N	N
Jiang et al. (2016), Jiang et al. (2018), Chen et al. (2020)	Y	N	DU	N	N	N	Y
Porwal and Mittal (2019a)	Y	Y	DU	N	Y	Y	N
Porwal and Mittal (2019b)	Y	Y	DU	N	Y	Y	N
Xu et al. (2019)	Using key update	N	DU	N	N	N	N
Ma et al. (2019), Lian et al. (2020), Zhang et al. (2021)	N	N	-	N	Y	Y	N
Liu et al. (2019), Premkamal et al. (2021)	N	N	–	N	Y	Y	Y
Cao and Markowitch (2020)	Y	N	DU	N	N	N	N
Ali et al. (2020a)	Y	N	DU	N	Y	Y	N
Cui et al. (2020)	Y	N	DU	N	N	N	Y
Ali et al. (2020b)	Y	Y	DU + AAs	N	N	N	N
Huang et al. (2018), Wang et al. (2017)	Y	N	DU	N	N	N	N
Ahuja and Mohanty (2017)	Y	Y	DU	N	N	N	N
Pussewalage et al. (2017)	Y	Y (Time Limit)	DU	N	N	Y	N
Xu et al. (2015)	Y	Y (Count limit)	DU + AA	Y	Y	N	N
The proposed work	Y	Y (Count limit)	DU	N	Y	Y	Y

3 Preliminaries

This section describes the cryptographic basics and other terminology used in design of the proposed scheme. This section describes Bilinear mapping, Monotonic access structure, access policy and the DBDH assumption.

3.1 Bilinear mapping

Assume that there are two cyclic group $\mathbb{G}_0$and$\mathbb{G}_1$ of prime order p and the generator of $\mathbb{G}_0$ is $\mathscr{g}$. A mapping function e defined as $e : \mathbb{G}_0 \times \mathbb{G}_0 \rightarrow \mathbb{G}_1$ is a bilinear mapping function if it satisfies the following properties.

Bilinearity: $\forall x, y \in \mathbb{G}_0$ and $\forall a, b \in Z_p$ mapping function satisfies-$e\left(\mathscr{g}^a, \mathscr{g}^b\right) = e(\mathscr{g}, \mathscr{g})^{ab}$
Non-degeneracy: Mapping $e\left(\mathscr{g}, \mathscr{g}\right) \neq 1$
Efficiently Computable: $\forall x, y \in \mathbb{G}_0$ mapping function e defined as $e : \mathbb{G}_0 \times \mathbb{G}_0 \rightarrow \mathbb{G}_1$ should be able to compute efficiently.

3.2 Monotonic access structure

If $U = \{U_1, U_2, \ldots, U_n\}$ be a set of attributes then an access structure $A \subseteq 2^{\{U_1, U_2, \ldots, U_n\}}$ is monotonic if for any $Y, Z \in 2^{\{U_1, U_2, \ldots, U_n\}}$, if $Y \in A$and$Y \subseteq Z$then$Z \in A$. For example, if $U = \{a_1, a_2, a_3, a_4\}$ then $(\{a_1, a_2, a_3\}$, $\{a_1, a_2, a_4\}$, $\{a_1, a_3, a_4\}$, $\{a_2, a_3, a_4\}$, $\{a_1, a_2, a_3, a_4\})$ are examples of monotonic access structures.

3.3 Access Policy And Access Tree

In CP-ABE technique, access policy P is defined by the data owner at the time of encryption and embedded with the ciphertext. This access policy P is generated by combining attributes of user, using AND, OR and threshold gate (*kofn*).

Access policy P is stored in the form of monotonic access tree structure $\mathbb{A}$ where leaf nodes represent the users' attribute and internal nodes represent the AND, OR and threshold gates (*kofn*). $\mathbb{A}_x$ represents a subtree of $\mathbb{A}$ rooted at x such that $parent(x)$ returns parent of x,$index(x)$ returns the position of x in ordered sequence of nodes in $\mathbb{A}$ and $attr(x)$ returns the value of attribute stored in leaf node$x$. Apart from these following terms are also defined:

num_x. It represents the number of the node x's children in T.
k_x. It denotes the threshold value of node x, where $0 < k_x \leq num_x$.

If node x is a non-leaf node and $k_x = 1$, x is an $\varepsilon OR\varepsilon$ gate. If node x is a non-leaf node and $k_x = num_x$, x is an $\varepsilon AND\varepsilon$ gate. For each leaf node, we define the threshold value $k_x = 1$.

An access policy with 2of2 threshold would be maximum length that requires computing all attributes of access policy attached with ciphertext at decryption. In results section, such access policy is written as $attr_1 attr_2 2of2 attr_3 2of2 \ldots attr_n 2of2 attr_n 2of2$. If we represent this policy in access tree then there will be $n-1$ internal nodes at different levels. Hence, with this access policy maximum $n-1$ access keys can be attached.

3.4 Decisional bilinear Diffie Hellman assumption

The challenger chooses two groups $\mathbb{G}_0$and$\mathbb{G}_1$ of prime order p that satisfy properties of bilinear function and defines a mapping function $e : \mathbb{G}_0 \times \mathbb{G}_0 \rightarrow \mathbb{G}_1$ which is efficiently computable. The challenger randomly chooses a generator $\mathscr{g} \in \mathbb{G}_0$, integer $a, b, c \in Z_p$ and $K \in \mathbb{G}_1$. Using the tuples $< g, \mathscr{g}^a, \mathscr{g}^b, \mathscr{g}^c >$, the adversary must distinguish between the tuple $< e(\mathscr{g}, \mathscr{g})^{abc} >$ and K.

A simulator *Sim* gives a guess $v \in \{0, 1\}$ in probabilistic polynomial time with advantage ε in solving DBDH in $\mathbb{G}_0$, if Eq. (1) was satisfied.

$$\left| \begin{array}{l} Probability\left[Sim\left(Res = e\left(\mathscr{g}, \mathscr{g}\right)^{abc}\right) = 0\right] - \\ Probabilty[Sim(Res = K) = 0] \end{array} \right| \geq \varepsilon \tag{1}$$

4 A fully flexible key delegation mechanism with fine grained access control system

This section gives the threat model, notations, important data structures, system model, functions and overview of the proposed fully flexible key delegation mechanism with fine grained access control. Mainly five entities are part of the system and their roles are described as below:

Central Server/Cloud server (CS): CS is a device that has unlimited storage capacities. Data users (DUs) upload encrypted content for access.

Central Authority (CA): CA generates system parameters PP, MSK and is responsible for distributing secret keys to DUs. The secret key for the user I_i having set of attributes S are generated in two parts say $SK_{S,I_i,1}$ and $SK_{S,I_i,2}$. CA stores $SK_{S,I_i,1}$ into TL, which is accessible to the trusted mediator and sends $SK_{S,I_i,2}$ to data users.

Trusted Mediator (TM): TM acts as a checkpoint for controlled key delegation. TM provides protection

against unauthorized key delegation and traces the delegator. TM is also responsible to protect against unauthorized decryption.

Data Owner (DO): Data owner generates content and different access keys, encrypts these access keys using the proposed scheme and uploads the generated ciphertext embedded with a single access policy on Central Server/ Cloud Server.

Data Users (DUs): DUs can access the encrypted content access keys uploaded on the central server if his/her attributes satisfy the access policy. If DU is a delegator, then he/she can access authorized rights any number of times but if DU is a delegatee then he/she can access delegated content only limited times, as specified by the delegator.

4.1 Threat model

It is assumed that central authority (CA) is an entity fully trusted by all. In the traditional key delegation mechanism, any user can create a delegation secret key on its own and share it with the delegatee [Bethencourt]. However, it can lead to uncontrolled delegation and possibility of system abuse. Thus, in this work, constrained key delegation is being sought. Some authority has to monitor the delegations. This task can't be done by CA as it is already overwhelmed with user management. This imposes the requirement of a third party, Trusted Mediator (TM), for mediated key delegations and access of content. TM verifies delegator's capability and delegatee's access count before allowing delegated access. It is trusted to honestly perform all designated tasks. However, being a third party, it is to be ensured that it itself cannot access/ decrypt any user's data on its own by using delegated keys. Thus, TM has been considered as a semi-trusted entity and this threat from TM has to be addressed in the proposed scheme. The scheme has been designed in such a way that there is no threat from compromised or corrupt TM.

Cloud server (CS) and data users (DUs) have no mutual trust. Therefore, user's data is to be stored in encrypted form to protect from unauthorized access by cloud servers as well as other users. Apart from this, some users may try to collude to gain access with attribute sets possessed by either of them. For key delegation, it is assumed that the delegator is honest and does not collude with delegatee to attack the system. It is also assumed that the data owner is keen to protect his data and thus changes the access key after completion of each delegation task.

4.2 Notations

Table 2 shows the description of symbols and notations used in construction of the proposed scheme.

4.3 System model

System model represents the flow of information among various entities CS, CA, TM, DO, and DUs. CA generates public parameter (PP) and master secret key (MSK) and shares PP with all the users but keeps MSK secret.

As shown in Fig. 2, To get the secret key, a data user i sends his/her identity I_i and set of attributes S to CA.CA generates secret key SK_S for data user$i$, having attribute set$S$, into two components as$SK_S = \{(SK_{S,I_i,1}), (SK_{S,I_i,2})\}$.. CA stores $(SK_{S,I_i,1})$ into TL and sends $(SK_{S,I_i,2})$ to data user$i$. DO encrypt different access keys for read, write and update operations on content$m$, generates integrated ciphertext CT for different access keys and stores CT on the CS. When a DU i needs to transmit his/her partial access rights to another data user$j$, he/she takes the identity I_j of DU j and generates$SK_{S,I_j,2}, S_{S\to\widehat{S}}$. Here, $SK_{S,I_j,2}$ is a part of decryption key and $S_{S\to\widehat{S}}$ is the transformation key. DU i sends $SK_{S,I_j,2}$ to DU j and identity I_i and transformation key $S_{S\to\widehat{S}}$ of attributes $\widehat{S}$ to TM. Later, TM accesses TL and verifies if DU i is allowed for further delegation or not. If DU i is allowed then TM updates TL and CL. To decrypt the ciphertextCT, DU j sends to TM, the new attribute sets $\widehat{S}$ and content m he/she wants to access. TM partially decrypts the CT and provides intermediate ciphertext $\widehat{CT}$ to DUj. DU decrypts $\widehat{CT}$ using $SK_{S,I_j,2}$ and gets different access keys of m. Finally, DU j accesses content m in the mode for which he/she got the access key.

As shown in.

4.4 Cryptographic functions

The important functions are implemented for the working of proposed scheme:

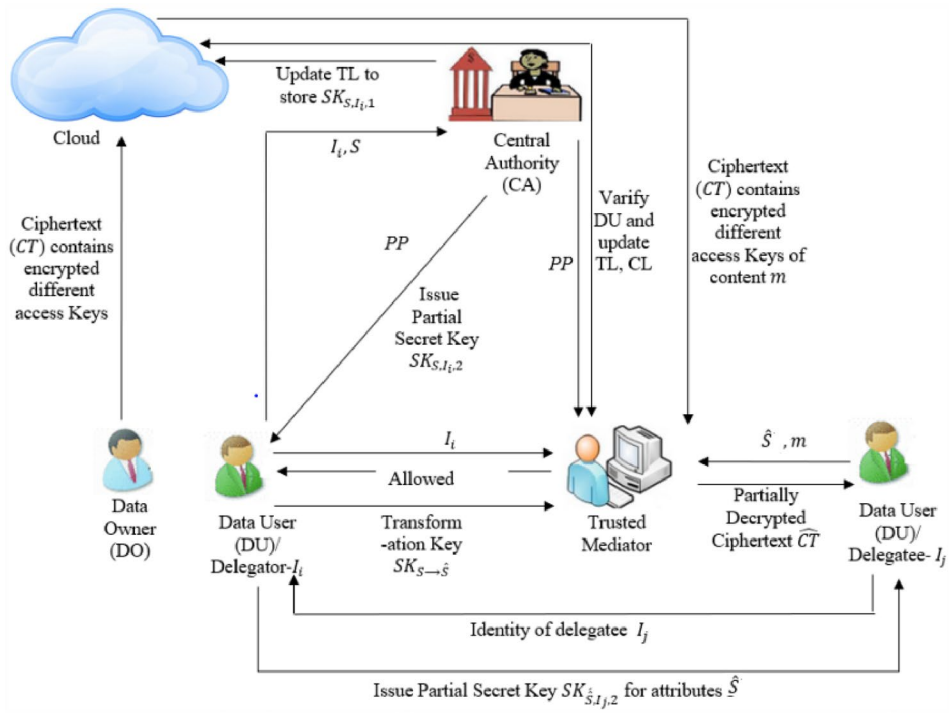
$Setup(\mathcal{U}) \to (PP, MSK)$: The function takes as input a universe set $\mathcal{U}$ of all possible attributes. CA executes the function and generates public parameters PP accessible to all entities and keeps master secret key MSK with itself. TL, CL, IRL and ARL are also created by this function.

$KeyGen(PP, MSK, S, I_i) \to ((SK_S) = \{(SK_{S,I_i,1}), (SK_{S,I_i,2})\})$: The function takes input public parameter PP, CA's master secret key MSK, an attribute set S and identity I_i of data user i. The function is executed by CA and generates data user decryption key $SK_S = \{SK_{S,I_i,1}, SK_{S,I_i,2}\}$. CA creates a tuple $< I_{CA}, H(I_i), SK_{S,I_i,1}, < 0, \infty >, ALL >$

Table 2 List of symbols and notations

Notations	Description
$\mathbb{G}_0, \mathbb{G}_1$	Bilinear group of prime order p
$\mathscr{g}$	Generator of bilinear group $\mathbb{G}_0$
Z_p	Set of Integer modulo p
$\mathbb{A}$	Access Tree
$\mathcal{U}$	Universal set of original attributes of size n
CA	Central Authority
TM	Trusted Mediator
CS	Cloud Server
DU	Data User
H	Hash function (Available publicly)
Γ	Space for user identity
a_i	i^{th} attribute
$\alpha, \beta, r_1, .., r_{n+l}$	Random numbers $\in Z_p$
S	Set of elements in Z_p
$\Delta_{i,S}(s)$	The Lagrange coefficient where, $i \in Z_p$
PP	Public Parameter
MSK	Master secret key
I_i	Identity of data user i
S	Set of attributes of original data user
$SK_{S,I_i,1}$	First part of secret key of I_i generated using S
$SK_{S,I_i,2}$	Second part of secret key of I_i generated using S
CL	Capability list store the access rights for delegated user
TL	Trace list store the delegation rights for every user
Key_S	Symmetric secret key for CA and TM only
P	Access policy
$Count_{Del_Curr}$	Value of count of current delegation
$Count_{Del_Max}$	Value of count of maximum delegation
Ptr_to_CL	Pointer to capability list
$\widehat{S}$	Set of attributes of delegated data user
M	Array of delegated content names
$SK_{S \to \widehat{S}}$	Transformation key
$Count_{Read_Max_m}$	Maximum limit to access the read key of content m for delegated user
$Count_{Read_Curr_m}$	Current Count of read key of content m access
$Count_{Write_Max_m}$	Maximum limit to access the write key of content m for delegated user
$Count_{Write_Curr_m}$	Current Count of write key of content m access
$Count_{Update_Max_m}$	Maximum limit to access the update key of content m for delegated user
$Count_{Update_Curr_m}$	Current Count of update key of content m access
Ptr_to_nextCL	Pointer to next record in CL
CT	Final Ciphertext generated by the proposed scheme
key_{Read}	Key to access a file for read
key_{Write}	Key to access a file for write
key_{Update}	Key to access a file for update
$\widehat{CT}$	Intermediate ciphertext generated by mediator
$Result_{Del}$	Store Boolean value true if the user is allowed to further delegatee
ALL	Value in Ptr_to_CL for the original user
Key_{T_C}	Common encryption/decryption key for CA and TM to encrypt and decrypt TL and CL
M	Array contains names of delegated contents
CTR_{DEL}	Maximum number of delegations
CTR_R	Array contains maximum reads for delegated contents
CTR_W	Array contains maximum writes for delegated contents
CTR_U	Array contains maximum updates for delegated contents

Fig. 2 System model



for user i and stores into TL and sends second component of decryption key $\left(SK_{S,I_i,2}\right)$ to data user i. CA stores ALL in Ptr_to_CL for original user in trace list (TL) indicate that this original user can access any file whose access policy is satisfied by the users' attribute. CA encrypts TL using Key_S and stores it on the cloud. TL contains the maximum number of delegations the user is allowed.

$Delegate(I_i, SK_{S,I_i,2}, \hat{S}, I_j) \rightarrow \left(SK_{S,I_j,2}, S_{S\to\hat{S}}\right)$: The Delegator i (I_i) executes this function to delegate a key for attribute set $\hat{S}$ to another data user j having identityI_j. The function outputs the second component of secret key $SK_{\hat{S},I_j,2}$ for j. Delegation rights $S_{S\to\hat{S}}, M, CTR_{DEL}, CTR_R, CTR_W, CTR_U$ are sent to TM to update TL and CL. This function also sends I_i, S''_{I_i}, I_j to TM.

$TM_Delegate(SK_{S,I_i,1}, SK_{S\to\hat{S}}, I_i, \hat{S}, I_j, Ctr_{Del}, M, CTR_R, CTR_W, CTR_U) \rightarrow SK_{S,I_j,1}$: This function is executed by TM and it takes first component of secret key $SK_{S,I_i,1}$ of delegator and a transformation key $SK_{S\to\hat{S}}$ and computes first component of secret key $SK_{\hat{S},I_j,1}$ of delegatee. Here M is array of content names and $Ctr_{Del}, CTR_R, CTR_W, CTR_U$ are maximum limits of delegation, read access, write access, update access of respective contents.TM generates new tuples for M and stores in TL and CL.

$Encrypt(KEY, \mathbb{A}, PP, m) \rightarrow CT$: The function takes input an array of keys KEY, access tree structure $\mathbb{A}$ and public parameter PP and generates single ciphertext CT. Assume that data owner shares a file m with different access keys i.e. $KEY = \{key_1, key_2, key_3\}$, where key_1 is key_{update}, key_2 is key_{write} and key_3 is key_{read}.

$TM_Decrypt(CT, \hat{S}, SK_{S,I_i,1}, m, Access_Mode) \rightarrow \widehat{CT}$: This function is executed by TM. The input required by function are ciphertext CT, extended set of attributes $\hat{S}$ of data user I_i who wants to decrypt the message and first component of secret key $SK_{S,I_i,1}$. The function also takes content name m and $Access_Mode$ and computes $\widehat{CT}$ if attributes of I_i satisfies the access tree structure for required accessing modes and access limit does not exceed otherwise set $\perp$ in $\widehat{CT}$.

$Decrypt\left(\widehat{CT}, SK_{S,I_i,2}\right) \rightarrow KEY$: This function is executed by data user who wants to access the content. On receiving array $\widehat{CT}$, the function decrypts $\widehat{CT}$ from $SK_{S,I_i,2}$ and generates required plaintext, that is, the access keys KEY.

4.5 Proposed data structures

The proposed scheme defines and uses following important data structures which are created and manipulated using indifferent cryptographic functions to implement the proposed scheme.

4.5.1 Trace List (TL)

Trace List (TL) has been designed to store the tuple for each original and delegated user. It is created by the CA during system initialization using *Setup*() function. It is updated in *KeyGen*() by CA and by TM in *TM_Delegate*() to limit the number of further delegations. This data structure is used to keep track of whether a particular user is allowed to further delegate or not. Each tuple in this list stores following information:

I_i : Identity of delegator or key generator i.
I_j : Identity of delegatee or user j, who receives the key.
$SK_{S,I_j,1}$: First component of secret key for attribute set S and for user j.
$Count_{Del_Curr,I_j}$: Count of current delegation done by j.
$Count_{Del_Max,I_j}$: Count of maximum delegation allowed to j.
Ptr_to_CL: Pointer to capability list.

The TL is accessible to CA and TM only. If a user i with identity I_i and set of attribute S is an original user then entry in TL will be $< I_{CA}, H(I_i), SK_{S,I_i,1}, < 0, Count_{Del_Max,I_i} >, Ptr_to_CL >$, where, $SK_{S,I_i,1}$ is the first part secret key of I_i, $Count_{Del_Max,I_i}$ is set to 0 and $Count_{Del_Max,I_i}$ is the maximum delegation limit of i, Ptr_to_CL is pointer to capability list having different accessing rights for i. Here Ptr_to_CL contains *ALL* because I_i is the original user and he/she can access all those contents, whose access policy is satisfied by its attributes. If a user i (with identity I_i) is delegated user and received key from k (with identity I_k) and delegated set of attribute $\widehat{S}$ the entry in TL will be as $< I_k, H(I_i), SK_{\widehat{S},I_i,1}, < 0, Count_{Del_Max,I_i} >, Ptr_to_CL >$, here, $SK_{S,I_i,1}$ is the first part of delegated secret key and $Count_{Del_Max,I_i}$ is the maximum delegation limit of i and Ptr_to_CL is pointer to delegation capability list.

4.5.2 Capability List (CL)

The purpose of this list is to provide fully flexible delegation which keeps track of specific delegated contents, their access rights and access limit to a delegatee. Capability list (CL) is created by CA at system initialization using *Setup*() function and updated by TM while executing function *Delegate*() to implement the proposed fine grained key delegation. This list stores the tuples for each delegated user and records following details:

- m: content name
- $Count_{Read_Curr,I_i}$: Count of read access of content m done by user with identity I_i
- $Count_{Read_Max,I_i}$: Count of maximum read access of content m allowed to user with identity I_i
- $Count_{Write_Curr,I_i}$: Count of write access of content m done by user with identity I_i
- $Count_{Write_Max,I_i}$: Count of maximum write access of content m allowed to user with identity I_i
- $Count_{Update_Curr,I_i}$: Count of update access of content m done by user with identity I_i
- $Count_{Update_Max,I_i}$: Count of maximum update access of content m allowed to user with identity I_i
- Ptr_to_nextCL: This pointer points to the next tuple if other contents are delegated otherwise, it contains *NULL*.

Only CA and TM can access and modify this list. If a user i with identity I_i is a delegated user then the entry in CL will be as follows:

$$< m, < Count_{Read_Curr,I_i}, Count_{Read_Max,I_i} >, Count_{Write_Curr,I_i}, Count_{Write_Max,I_i} >, < Count_{Update_Curr,I_i}, Count_{Update_Max,I_i} >, Ptr_to_nextRL >$$

where m is the name of content, $< Count_{Read_Curr,I_i}, Count_{Read_Max,I_i} >$ indicates < count of current read of I_i, maximum read limit allowed to I_i>, $< Count_{Write_Curr,I_i}, Count_{Write_Max,I_i} >$ indicates < count of current writes by I_i, maximum write limit allowed to I_i>, and $< Count_{Update_Curr,I_i}, Count_{Update_Max,I_i} >$ indicates < count of current update of I_i, maximum update limit allowed to I_i> and Ptr_to_nextRL points to next tuple if other contents are delegated otherwise it contains *NULL*.

4.5.3 Revocation Lists (RLs)

The purpose of revocation lists (RLs) is to protect unauthorized access of contents by revoked delegatee. The proposed system maintains two revocation lists. One is Attribute Revocation List (ARL) that contains the set of attributes revoked by the user or system. If the decryptor's attributes set has an attribute stored in ARL then the decryptor is not allowed to decrypt any content. Another is Identity Revocation List (IRL), which contains a hash of the identity of revoked users. If the hash of decryptor's extended set of attributes matches with hash of identity of revoked users

stored in IRL then decryptor is not allowed to decrypt any content. CA creates IRL and ARL simultaneously during system initialization using *Setup*() function. TM accesses IRL and ARL in function *TM_Decrypt*() to check if a data user is revoked fully or partially.CA updates IRL when it revokes a data user completely through its identity. IRL is mainly updated when a data user leaves the system. On the other hand, CA updates the ARL when it revokes only a set of attributes and not the user completely.

5 Construction of the proposed scheme

In this section, main cryptographic functions designed to implement the scheme are constructed. Let $\mathbb{G}_0$, $\mathbb{G}_1$ be two bilinear groups of prime order p, and let generator of group $\mathbb{G}_0$ be g. A bilinear mapping function is denoted as $e : \mathbb{G}_0 \times \mathbb{G}_0 \rightarrow \mathbb{G}_1$. The size of the group is determined by the security parameter λ. For secret sharing, the langrage coefficient is defined as $\Delta_{i,S}(s) = \prod_{j \in S, j \neq i} \frac{s-j}{i-j}$, where $i \in Z_p$ and S is a set of elements in Z_p.

A universal set with n original attributes is defined as $= \{a_1, \ldots, a_n\}$, each user is assigned attributes out of this set. The scheme also makes use a hash function namely, $H : \{0,1\}^* \rightarrow \{0,1\}^l$ to map the identities of users.

Function 1 $Setup\left(\mathcal{U}, \mathcal{U}'\right) \rightarrow (PP, MSK)$:

This function is executed by the central authority (CA). The function then chooses random numbers $\alpha, \beta, r_1, .., r_{n+l} \in Z_p$ and random group generator $g \in \mathbb{G}_0$ to compute PP, MSK as given in Eq. (2) and (3).

$$PP = \{\mathbb{G}_0, g, h = g^{\beta}, f = g^{1/\beta}, Y = e\left(g, g\right)^{\alpha}, h_1 = g^{r_1}, \ldots, h_n = g^{r_n}\} \tag{2}$$

$$MSK = \{\beta, g^{\alpha}, r_1, .., r_n\} \tag{3}$$

TL, CL, IRL and ARL are also initialized in this function.

Function 2 $KeyGen\left(PP, MSK, S, I_i\right) \rightarrow ((SK_S) = \{\left(SK_{S,I_i,1}\right), \left(SK_{S,I_i,2}\right)\})$:

This function is executed by the central authority (CA). It takes input public parameter PP, master secret key MSK, set of attributes S and data users' identity I_i. The function chooses a random number $r \in Z_p$ is chosen and generates data user's decryption key components $\left\{SK_{S,I_i,1}, SK_{S,I_i,2}\right\}$ as per Eq. (4) and (5).

$$SK_{S,I_i,1} = \{D = g^{(\alpha+r)/\beta}\} \tag{4}$$

$$SK_{S,I_i,2} = \{S, \forall a_i \in SD_i' = g^{r}.H_1\left(a_i\right)^{r_i}, D_i'' = g^{r_i}\} \tag{5}$$

Then the function stores the tuple $< I_{CA}, H\left(I_i\right), SK_{S,I_i,1}, < 0, Count_{Del_Max} >, Ptr_to_CL >$ into trace list (TL) of trace and sends decryption key $\left(SK_{S,I_i,2}\right)$ to user i.

Function 3 $Delegate(I_i, SK_{S,I_i,2}, \widehat{S}, I_j) \rightarrow \left(SK_{S,I_j,2}, S_{S\rightarrow\widehat{S}}\right)$:

This function is executed by the data user for delegation. The function takes as input identity I_i of delegator i, part of secret key $SK_{S,I_i,2}$, attributes set $\widehat{S} \subseteq S$ and delegatee's identity I_j.

The second component of the decryption key of delegatee, $SK_{\widehat{S},I_j,2}$ is generated as per Eq. (6) and sent to delegatee.

It sends the identity of the delegator I_i, the identity of the delegatee I_j and set of attributes $\widehat{S}$ to be delegated to TM. TM accesses TL and checks further delegation rights of I_i. If I_i is authorized to delegate then it generates another component of decryption key by choosing the random numbers $\widehat{r}, \widehat{r}_j \forall a_j \in \widehat{S}$ from Z_p as follows:

$$SK_{\widehat{S},I_j,2} = \{\widehat{S}, \forall a_j \in \widehat{S}\widehat{D}_j' = D_j' g^{\widehat{r}}.H\left(a_j\right)^{\widehat{r}_j}\} \tag{6}$$

Then, it computes the transformation key as Eq. (7).

$$S_{S\rightarrow\widehat{S}} = f^{\widehat{r}} \tag{7}$$

Transformation key$S_{S\rightarrow\widehat{S}}$, number of further delegations inCtr_{Del}, delegated content names in array $[]m$ in M and their read/write/update counts$[]Ctr_{Read}$, $[]Ctr_{Write}$ and $[]Ctr_{Update}$ inCTR_R, CTR_W and CTR_U for updating TL and CL are sent to TM.

Function 4 $TM_Delegate(SK_{S,I_i,1}, SK_{S\rightarrow\widehat{S}}, I_i, \widehat{S}, I_j, Ctr_{Del}, M, CTR_R, CTR_W, CTR_U) \rightarrow \widehat{S}, , SK_{\widehat{S},I_j,1}$:

TM receives the identity I_i and set of attributes $\widehat{S}$ of delegator and checks the trace list (TL) to decide whether delegator I_i is authorised to delegate his/her key or delegated key. TM searches $<$TL$>$ for the tuple $< I_k, I_i >$ if there exists a tuple for I_i and $I_{CA} = I_k$, it means I_i is an original data user and not a delegatee. TM updates TL and CL as follows:

If $Count_{Del_Curr} + 1 > Count_{Del_Max}$ then set $Result_{Del} = False$ else set $Count_{Del_Curr} = Count_{Del_Curr} + 1$ into TL for I_i. If I_i is already delegated user and received the key from I_k then repeat the same procedure until the original data user's count is increased. The first component of decryption key for I_j to access the contents delegated by I_i is computed as given in Eq. (8).

$$SK_{\widehat{S},I_j,1} = SK_{S,I_i,1}.SK_{\widehat{S}\rightarrow\widehat{S}} \tag{8}$$

It creates a tuple of TL for delegatee $< I_i, I_j, SK_{\widehat{S},I_j,1}, < 0, \min((Count_{Del_Max} - Count_{Del_Curr}), Ctr_{Del}) >, Ptr_to_CL >$, appends it to existing TL and set $Result_{Del} = True$ and sends $Result_{Del}$ to I_i.

It then creates the tuples $rl_1, rl_2, \ldots, rl_k$ for the contents to be delegated and appended to CL. For example, $< m_1, < 0, \min((Count_{Read_Max_m1} - Count_{Read_Curr_m1}), Ctr_{Read_m1})$

$>, < 0\min((Count_{Write_Max_m1} - Count_{Write_Curr_m1}), Ctr_{Write_m1}) >$, $< 0, \min((Count_{Update_Max_m1} - Count_{Update_Curr_m1}), Ctr_{Update_m1}) >$, $Ptr_to_nextRL >, < m_2 < 0, , \ldots\ldots\ldots < m_k, < 0,$ describes the delegated access rights. If a content m_i is already delegated by I_l to I_j then count for read/write/update will be increased to the existing access rights ofI_l.

Function 5 $Encrypt(KEY, \mathbb{A}, PP) \rightarrow CT$

Assume that data owner shares a content m with k access keys i.e. $KEY = \{key_1, key_2, key_3\}$. For instance, key_1 is key_{Update}, key_2 is key_{Write} and key_3 is key_{read} and their priorities are like $key_{Update} \geq key_{Write} \geq key_{Read}$.

It selects 3 random numbers $s_1, s_2, s_3 \in Z_p$ and computes $\tilde{C}_i$ and $C_i \forall i = 1, 2, 3$ as given in Eq. (9) and (10).

$$\tilde{C}_i = key_i.e(\mathscr{g}, \mathscr{g})^{s_i} \forall i = 1, 2, 3 \tag{9}$$

$$C_i = h^{s_i} \forall i = 1, 2, 3 \tag{10}$$

Then it chooses a polynomial $q_{(x,y)}(z)$ for each non- leaf and leaf node (x, y) in access tree structure $\mathbb{A}$. These polynomials are chosen in following manner, starting from the root node R.

Set degree of the polynomial $d_{(x,y)} = k_{(x,y)} - 1 \forall (x, y) \in \mathbb{A}$, where $k_{(x,y)}$ is the threshold value.

Set $q_R(0) = s_1$ and chooses d_R more points to completely define polynomial $q_R(z)$.

Set $q_{(x,y)}(0) = s_i \forall i = 1, 2, 3$, if $(x, y) \in AccessNode$ otherwise set $q_{(x,y)}(0) = q_{parent(x,y)}(index(x, y))$. Further, $d_{(x,y)}$ points of $q_{(x,y)}$ are made of the level nodes of children of (x, y) while points of remaining nodes are randomly selected.

$\forall (x, y) \in Y$, where Y is set of leaf nodes, then it computes $C_{(x,y)}$ and $C'_{(x,y)}$ as given in Eqs. (11) and (12).

$$C_{(x,y)} = \mathscr{g}^{q_{(x,y)}(0)} \forall (x, y) \in Y \tag{11}$$

$$C'_{(x,y)} = H_1(att(x, y))^{q_{(x,y)}(0)} \forall (x, y) \in Y \tag{12}$$

$\forall child_j \in TN_CT$, where TN_CT are those children of X that are Threshold gate and X is the set of transport node, $\hat{C}_{(x,y),j}$ is computed as per Eq. (13).

$$\hat{C}_{(x,y),child_j} = e\left(\mathscr{g}, \mathscr{g}\right)^{\alpha.\left(q_{(x,y)}(0) + q_{child_j}(0)\right)}.H_2(e\left(\mathscr{g}, \mathscr{g}\right)^{\alpha.\left(q_{(x,y)}(0)\right)}) \tag{13}$$

Then integrated ciphertext as given in Eq. (14) is the final output.

$$CT = \left\{\mathbb{A},\ \tilde{C}_i, C_i, C_{(x,y)}, C'_{(x,y)}, \hat{C}_{(x,y),child_j}\right\} \tag{14}$$

Function 6 $TM_Decrypt(CT, S, SK_{S,I_i,1}, m_l, Access_Mode) \rightarrow \widehat{CT}$:

This function is executed by the trusted mediator. The function receives a set of attributes S from decryptor. It first checks ARL and IRL for revoked users. If the requested decryptor is not revoked then it checks TL to determine if the decryptor is the original user or delegated user.

If the decryptor is a delegated user then the following two additional steps are performed.

It checks the trace list (TL) to decide the decryptor is authorized to access the particular content and provides the intermediate ciphertext for only those different access mode keys for which he/she is authorized Later $<$TL$>$ is searched for the tuple $< I_k, I_i >$.

If $I_{CA} \neq I_k$ then I_i is confirmed to be a delegatee add it searches for particular content m_l in CL. If m_l exists in CL for I_i then find current count $Count_{key_i_Curr}$ and maximum count $Count_{key_i_Max}$ of the requested key_i where $i \in \{read, write, update\}$.If$Count_{key_i_Curr} + 1 > Count_{key_i_Max}$ then the $Result_{Access_i} = False$ else set $Count_{key_i_Curr} = Count_{key_i_Curr} + 1$ into RL for I_i. Since I_i is already delegated user and received the key from I_k then repeat the same procedure for I_k until the new I_k is an original data user. Finally, set $Result_{Access_i} = True$. When value of $Result_{Access}$ becomes $True$ then following steps are performed.

Let $i = Access_Mode_i$ of required key $Key_{Access_Mode_i}$ compute F_i as given in Eq. (15).

$$\begin{aligned} F_i &= e\left(\ C_i,\ SK_{S,\ I_i,\ 1}\right) \text{ where } i = Access_{Mode\ i} \\ &= e\left(h^{s_i}, g^{(\alpha+r)/\beta}\right) \text{ where } i = \ Access_Mode_i \end{aligned} \tag{15}$$

Generate intermediate ciphertext as given in Eq. (16).

$$\widehat{CT} = \{\mathbb{A}, \tilde{C}_i, F_i, C_{(x,y)}, C'_{(x,y)}, \hat{C}_{(x,y),child_j}\} \tag{16}$$

Send $\widehat{CT}$ to decryptor I_i.

Send a message to DO to re-encrypt the content.

Function 7 $Decrypt\left(\widehat{CT}, SK_{S,I_i,2}\right) \rightarrow KEY$:

This function is executed by the data user I_i, it takes as input intermediate ciphertext $\widehat{CT}$ and the second component of decryption key $SK_{S,I_i,2}$ and generates required keys KEY. A recursive function $DecryptLeafNode\left(\widehat{CT}, SK_{S,I_i,2}, (x, y)\right)$

$\forall (x, y) \in LeafNode$ is defined using Eq. (17).

If $att(x, y) \notin S_{I_i}$ then the function returns NULL else it returns the value obtained by Eq. (17).

$$DecryptLeafNode\left(\widehat{CT}, SK_{S,I_i,2}, (x, y)\right) = \frac{e\left(D'_i, C_{(x,y)}\right)}{e\left(D''_i, C'_{(x,y)}\right)}$$

$$= \frac{e\left(\mathscr{g}^r.H_1\left(a_i\right)^{r_i}, \mathscr{g}^{q_{(x,y)}(0)}\right)}{e\left(\mathscr{g}^{r_i}, H_1(att(x, y))^{q_{(x,y)}(0)}\right)}$$

$$= e\left(\mathscr{g},\mathscr{g}\right)^{rq_{(x,y)}(0)} \tag{17}$$

Another recursive function *DecryptLeafNode* $\left(\widehat{CT}, SK_{S,I_i,2}, (x',y')\right)\forall(x,y) \in NonLeafNodeand\left(x',y'\right) \in childof(x,y)$ is defined as follows.

Let $S_{(x,y)}$ be an arbitrary $k_{(x,y)}sized$ child node set of (x,y). If there does not exist $S_{(x,y)}$ set in S_{I_i} then the function returns NULL else $\forall\left(x',y'\right) \in childof(x,y)$ function $DecryptNonLeafNode\left(\widehat{CT}, SK_{S,I_i,2}, \left(x',y'\right)\right)$ executes and returns the value $F_{(x',y')}$ obtained by Eq. (18).

$$F_{(x',y')} = \prod_{(x',y')\in S_{(x,y)}} F_{(x',y')}^{\Delta_{i,S'_{(x',y')}}(0)} \forall\left(x',y'\right) \in childof(x,y)$$

, where

$$i = index(x',y')$$

and

$$S'_{(x',y')} = index\left(x',y'\right) : (x',y') \in S_{(x,y)}$$

, where

$$F_{(x',y')} = e\left(\mathscr{g},\mathscr{g}\right)^{rq_{(x',y')}(0)} \tag{18}$$

Then the function executes $DecryptNode\left(\widehat{CT}, SK_{S,I_i,2}, (x,y)\right) where (x,y) \in root$ is defined as Eq. (19).

$$DecryptNode\left(\widehat{CT}, SK_{S,I_i,2}, (x,y)\right) = \begin{cases} DecryptNonLeafNode\left(\widehat{CT}, SK_{S,I_i,2}, \left(x',y'\right)\right) \\ \forall\left(x',y'\right) \in childof(x,y) and \\ \forall(x,y) \in NonLeafNode \\ DecryptLeafNode\left(\widehat{CT}, SK_{S,I_i,2}, (x,y)\right) \\ \forall(x,y) \in LeafNode \end{cases} \tag{19}$$

It calls $DecryptNode\left(\widehat{CT}, SK_{S,I_i,2}, (x,y)\right)\forall(x,y) \in LevelNode$ and stores the result in $A_i(iinrange1 \ldots 3)$ as follows:

$$A_i = DecryptNode\left(\widehat{CT}, SK_{S,I_i,2}, (x,y)\right)$$

$$e\left(\mathscr{g},\mathscr{g}\right)^{rq_{(x',y')}(0)}$$

Then it computes $e\left(\mathscr{g},\mathscr{g}\right)^{rs_i}$ for each authorized level node using Eq. (20) and store the result in B_i.

$$B_i = \frac{F_i}{A_i} i \in [1,2,3] \tag{20}$$

Then it computes $e\left(\mathscr{g},\mathscr{g}\right)^{rs_{i+1}}, \ldots, e\left(\mathscr{g},\mathscr{g}\right)^{rs_{i+k}}$ for each jth lower authorization level node ($j \in [1,2]$).

$$E_j = \frac{\widehat{C}_{(x,y),j}}{H_2(B_j)} \tag{21}$$

$$B_{j+1} = E_j / B_j \tag{22}$$

Then the authorized different access keys are computed using Eq. (23)

$$ck_i = \frac{\tilde{C}_i}{F_i} = \frac{key_i.e\left(\mathscr{g},\mathscr{g}\right)^{rs_i}}{e\left(\mathscr{g},\mathscr{g}\right)^{rs_i}} i \in [1,2,3] = key_i \tag{23}$$

6 Analysis of the proposed scheme

This section gives the security, features and complexity and performance analysis of the proposed scheme as compared to existing schemes.

6.1 Security analysis

Proposed scheme is designed to be secure against Chosen Plaintext Attacks (CPA), uncontrolled delegation and access of content, content access in unauthorized mode, user collusion attack, and unauthorized access by revoked user and untraceable key delegation abuse.

Considering the proposed model of the system, we assume an adverse user is a malicious user who can decrypt the ciphertext stored on CS using his/her attribute set or by colluding with other malicious users. The security of the proposed scheme is defined by the security games given below:

6.1.1 Chosen plaintext attack security

A security game between challenger *Chl* and adverse user *Adv* is described for selective chosen plaintext attack. Where a user gets the secret key *SK* from central authority (CA) by providing identity I_i of user i and his/her attribute set S and ciphertext *CT* has encrypted access keys and an access tree. An adverse user *Adv* creates a challenging access tree $\mathbb{A}^*$. The *Adv* requires different *SK* s such that no *SK* satisfies $\mathbb{A}^*$.

Init: An *Adv* generates a challenging structure $\mathbb{A}^*$ for multilevel access leys and sends to the *Chl*.

Setup: *Chl* runs *Setup* and gives *PP* to *Adv*.

Phase 1: The *Adv* queries *Chl* to run *KeyGeneration* for secret keys *SKs* corresponding to attribute sets $\{S_1, S_2 \dots S_k\}$ such that$\forall i = 1 \dots k, S_i \notin W$.

Challenge: The *Adv* creates two access keys $\{k_0, k_1\}$ having same length and sends to *Chl*. Then *Chl* randomly guess $v \in \{0, 1\}$ and performs encryption on access key k_v with challenging structure W and sends the final CT to *Adv*.

Phase 2: The *Adv* queries *Chl* to run *KeyGeneration* for secret keys *SKs* corresponding to attribute sets $\{S_{k+1}, S_{k+2} \dots S_n\}$ such that$\forall i = k+1 \dots n, S_i \notin W$.

Guess: *Adv* randomly guess v' of v.

Definition1: *The proposed scheme is selective CPA secure If an adverse user cannot break the system in polynomial time with negligible advantage in this game.* (Canetti et al. 2003).

The security of the proposed scheme against CPA attack is proved by proving the theorems given below.

Theorem 1. *If DBDH assumption holds then an adversary cannot break the proposed scheme using CPA, in polynomial time with non-negligible advantage.*

Proof: Assume that there exists an adverse user *Adv* who can break the proposed scheme with non-negligible advantage ε. A simulator *Sim* is constructed to perform operations for adversaries.

According to simulator, the Challenger *Chl* chooses two bilinear groups $\mathbb{G}_0$and$\mathbb{G}_1$ of prime order p, generator g and sets a mapping function $e : \mathbb{G}_0 \times \mathbb{G}_0 \to \mathbb{G}_1$. Then, *Chl* randomly selects binary value $v \in \{0, 1\}$ and integers $a, b, c \in Z_p$ independent of view of *Sim*. Based on binary value of v, *Chl* sets R. If $v = 0$ then, *Chl* sets $R = e(g, g)^{abc}$ else sets $R = K$ and defines tuples as $< g, A = g^a, B = g^b, C = g^c, R >$ and sends tuples to *Sim*. Now *Sim* plays as *Chl*. For simplicity, it is assumed that *Chl* encrypts single access key and generates challenge $\mathbb{A}^*$ and $\{\tilde{C}, C'\}$ according to *Encrypt* function.

Init: In this phase, an *Adv* generates challenge $\mathbb{A}^*$ of $W^*(W^* \subseteq \mathcal{U})$ for multilevel access keys and gives to *Sim*.

Setup: *Sim* selects random numbers $\alpha, \lambda_1, .., \lambda_n, \gamma_1, .., \gamma_l, \in Z_p$ and defines$\alpha = \alpha' + ab$.

Since $h_i = g^{r_i} \forall i \in \mathcal{U}''$, the *Sim* sets $r_i = \lambda_i \in Z_p \forall i \in \mathcal{U}$, $r_{n+i} = \gamma_i \in Z_p \forall i \in \mathcal{U}'$.

To generate public parameter PP, *Sim* computes values $h, h_i, \dots, h_i, Y$

$$h_i = g^{\lambda_i} \forall i \in \mathcal{U}$$

$$h_{n+i} = g^{\gamma_i} \forall i \in \mathcal{U}'$$

$$Y = e(g, g)^{\alpha'}.e(g, g)^{ab}$$

Finally, *Sim* passes PP to *Adv*.

Phase 1: *Adv* queries for the secret key SK and sends the set of attributes$S$, I_{Adv} to *Sim* such that S does not satisfy W but there must exist an attribute q in S but$q \notin W$. Then *Sim* selects such attribute$q$. Next *Sim* chooses a random number $r' \in Z_p$ and sets $r = r' - a$ and computes $D = g^{\frac{(\alpha+r)}{\beta}} = \frac{g^{\alpha}.g^{r}}{g^{\beta}} = \frac{g^{\alpha'+ab}.g^{r}}{g^{\beta}} = \frac{g^{\alpha'}.g^{a}.g^{b}.g^{r}}{g^{\beta}} = \frac{g^{\alpha'}.g^{a}.g^{b}.g^{r}}{g^{b}} = g^{\alpha'}.g^{a}.g^{r} = g^{\alpha'}.g^{a}.g^{r'-a} = g^{\alpha'+r'}$. Then *Sim* computes D'_i, D''_i as: $D'_i = g^{r'-a}.H1(i)^{r_i} = \frac{g^{r'}}{g^{a}}.H1(i)^{r_i} = \frac{g^{r'}}{A}.H1(i)^{r_i} \forall i \in S''$ and$D'_i = h^{r_i} = g^{b.r_i} = B^{r_i} \forall i \in S''$. Finally, *Sim* passes the $SK < S'', D, D_i, D'_i >$ toAdv. Adaptively repeat the step.

Challenge: *Adv* outputs two different access keys k_0, k_1 to *Sim*.*Sim* selects a random binary bit $v' \in \{0, 1\}$ and then executes *TM_Encrypt* and *Encrypt* functions and then outputs ciphertext $CT = \{\mathbb{A}^*, \tilde{C}_i, C_i, C_{(x,y)}, C'_{(x,y)}, \widehat{C}_{(x,y),child_j}\}$ where $\mathbb{A}^*$ is challenging access tree and $\tilde{C}, C'$ are computed as

$$\tilde{C} = k_{v'}.e(g, g)^{\alpha s} = k_{v'}.e(g, g)^{\alpha c} = k_{v'}.e(g, g)^{(\alpha'+ab)c} = k_{v'}.e(g, g)^{\alpha' c}.R$$

and $C' = g^s = g^c = C$. Finally, *Chl* gives CT to *Adv*.

Phase 2: This phase executes in same way as phase 1.

Guess: Now *Adv* guesses a random binary bit $v'' \in \{0, 1\}$. If $v'' = v'$ then *Sim* outputs *False* for guess $R = e(g, g)^{abc}$ else *Sim* outputs *True* for $R = K$. Assuming that *Adv* breaks the proposed scheme with advantage ε. If $R = e(g, g)^{abc}$ then *Sim* results correctly and *Adv* guesses v'' perfectly with probability $\varepsilon + \frac{1}{2}$.

$$P_{False} = \left| Probability[Sim\left(R = e\left(g, g\right)^{abc}\right) = 0] \right| = \varepsilon + \frac{1}{2}$$

Sim outputs *True* for guess $R = K$. Hence $v'' \neq v'$ holds with probability $\frac{1}{2}$.

$$P_{True} = |Probability[Sim(R = K) = 0]| = \frac{1}{2}$$

Finally, the advantage to *Sim* in the game is$= \frac{1}{2}P_{False} + \frac{1}{2}P_{True} - \frac{1}{2} = \frac{\varepsilon}{2}$

Thus, it can be seen that the advantage of an adversary is negligible.

6.1.2 Limited times delegation security model

In this model, a delegatee can further delegate only a specified number of times. Scheme detects a data user if he/she tries to cross the specified delegation limit. A security game between *Chl* and *Adv* is given for this specific security of the proposed scheme. The game is defined as follows:

Setup: *Chl* runs *Setup* and gives *PP* to *Adv*.

Phase1: *Adv* can repeatedly issue all types of queries as defined in the RCCA security game.

Challenge: Lets a user tries to delegate secret key $(j)^{th}$ time and *Adv* submit challenge as S^* to *Chl*. *Adv* executes *Delegate* on S^* and generates transformation key $SK_{S\to\hat{S}}{}^*$ sends $SK_{S\to\hat{S}}{}^*$ along with its identity I_{Adv} and identity of the delegatee $I_{Adv'}$ and *Chl* executes *TM_Delegate* and sends $Result_{Del}$ to *Adv*.

Output: *Adv* outputs a delegation key $\widehat{SK}^*$ generated by *Delegate* where $Result_{Del} = True$. *Adv* wins the game if j is greater than the specified limit.

Definition 2: *The proposed scheme supports limited time delegation if an adverse user has at most a negligible advantage in this game.*

The security of the proposed scheme against unlimited delegation is proved by proving the theorems given below.

Theorem 2: *The adversary has negligible advantage in the limited times delegation security game proposed in the given scheme with the assumption that the decisional q-Bilinear Diffie-Hellman assumption holds. Here q is the number of further delegation queries.*

Proof: Assume that there exists an adverse user *Adv* that attacks limitation times key delegation security of the proposed scheme. *Adv* communicates with simulator *Sim* as follow:

Setup: Simulator *Sim* chooses random integers $\alpha, \beta, r_1, .., r_{n+l} \in Z_p$ and $g, h, u, v, w \in \mathbb{G}_0$.*Sim* selects three hash functions H, H_1, H_2 then computes$e(g,g)^{\alpha}$, g^{β}, $g^{1/\beta}$ and$h_1 = g^{r_1}, \ldots, h_n = g^{r_n}$. Finally, *Sim* provides $PP = \{\mathbb{G}_0, g, h = g^{\beta}, f = g^{1/\beta}, Y = e(g,g)^{\alpha}, h_1 = g^{r_1}, \ldots, h_{n+l} = g^{r_n}\}$ to every user. After that *Sim* chooses two random integers x_U and $x_{TM} \in Z_p$ and generatesPUB_U,PRK_U, PUB_{TM}, PUB_{TM} and gives (PP, PUB_U, PUB_{TM}) to adversary*Adv*.

Query Phase: Adversary *Adv* adaptively issues queries **Corrupt.SK(j)** and **Corrupt.TK(j)**. Since Simulator *Sim* has the *MSK* and properly answers queries of *Adv*.

Challenge: Assume that this is the ith request for further delegation. *Adv* executes *Delegate* on $\hat{S}^*$ and generates transformation key $SK_{S\to\hat{S}}{}^*$ and provides it $SK_{S\to\hat{S}}{}^*$ with its identity I_{Adv} and identity of the delegatee $I_{Adv'}$ to *Sim*. *Sim* access TL and determines I_{Adv} is allowed to further delegate or not and sends $Result_{Del}$ to *Adv*.

Output:*Adv* is allowed to delegate if $Result_{Del} = True$. If the adverse user is allowed to delegate then TM check the condition $Del_{max} - Del_{curr} > 0$. An adversary *Adv* wins this game if $Del_{max} - Del_{curr} \leq 0$anditisallowedtodelegate.*Adv* wins with $i = Del_{max} + 1$. TM increases the counter Del_{curr} for delegatee $k_{Id_{Adv}}$ or another delegatee in TL. TM sends S''^* for $k_{Id_{Adv}}$ and increases Del_{curr} and the uniqueness property of $k_{Id_{Adv}}$ holds if the decisional *q-BDH* assumption holds. Since *Adv* is allowed to execute *Delegate* at most Del_{max} times, by using the concept pigeonhole principle, it shows that there are at least two identical values of $k_{Id_{Adv}}$. This contradicts the assumption and the TM rejects the outsourced decryption request. Hence,*Adv* wins this game with negligible probability.

Hence the theorem is proved.

6.1.3 *Theorem 3: The proposed scheme is collusion attack free.*

Proof. In the presented scheme, one component of the secret key $\mathrm{SK_S} = \{\mathrm{SK_{S,I_i,1}}, \mathrm{SK_{S,I_i,2}}\}$ is stored in TL and another component is provided to the user. Whenever the data user wants to access content, he/she sends his/her identity to TM. TM computes intermediate ciphertext using a secret key component $\mathrm{SK_{S,I_i,1}}$ and gives to the DU. The users do not have their complete secret key, hence there is no advantage in getting the secret key component from other users. Thus, the proposed scheme is collusion attack free.

6.1.4 *Theorem 4: The proposed scheme is secure against uncontrolled access of content.*

Proof. In the presented scheme, secret key SK is generated in two components $\mathrm{SK_S} = \{\mathrm{SK_{S,I_i,1}}, \mathrm{SK_{S,I_i,2}}\}$. $\mathrm{SK_{S,I_i,1}}$ is attribute independent component and is stored in TL, accessible to TM and other component $\mathrm{SK_{S,I_i,2}}$ is provided to the user. Whenever the data user wants to access content, he/she sends his/her identity to TM. TM accesses TL, CL and checks the access limit of data users for particular content. If data user is authorized then TM computes intermediate ciphertext using a secret key component $\mathrm{SK_{S,I_i,1}}$ and gives to the DU. Hence, data users can access only that content for which they have access limit otherwise TM will not generate intermediate ciphertext for the specific content.

6.1.5 *Theorem 5: The proposed scheme is secure against content access in unauthorized mode.*

Proof. To access a particular content in specific mode, the data user provides his/her identity to TM. The TM accesses TL, CL and checks the access limit of data users for particular content in specific mode. If a data user is authorized then TM updates CL and provides intermediate ciphertext to the DU. Hence, data users cannot access any content in unauthorized access mode.

6.1.6 *Theorem 6: The proposed scheme is secure against content access by revoked user.*

Proof. To access particular content, a data user provides his/her identity to TM. TM first accesses IRL and checks the

Table 3 Features comparison of proposed scheme with existing works

Delegation Scheme	Specific attribute delegation	Specific Content delegation	Specific access right delegation	Specific limit in further delegation	Specific limit for accessing of specific delegated content	Instant attribute revocation	User Revocation based on identity	Trace-ability
Basic delegation (Bethencourt et al. 2007)	Yes	No	No	No	No	No	No	No
Flexible further delegation (Porwal and Mittal 2019a)	Yes	No	No	Yes	No	Yes	Yes	Yes
Traceable delegation (Chen et al. 2020)	Yes	No	No	No	No	No	No	Yes
Proposed scheme	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

identity in IRL. If the data user has not been revoked then TM accesses ARL to check for any revoked attribute of this user. If a user's attributes related to the ciphertext have not been revoked then TM checks TL, CL and determines the access limit of the data user. If a data user is authorized then TM updates CL and provides intermediate ciphertext to the DU. Hence, revoked users cannot access any unauthorized content.

6.1.7 ***Theorem 7****: The proposed scheme is able to trace unauthorized further delegation.*

Proof. To delegate content, the data user *ii* provides his/her identity I_i, delegatee's identity, transformation key, further delegation limit count and read/write/update access limit counts to TM. TM first accesses TL to check the delegation limit of I_i by reading tuple $< I_k, I_i, SK_{S,I_i,1}, < 0, Count_{Del_Max_I_i} >, Ptr_to_CL >$. If current delegation count of I_i is equal to $Count_{Del_Max_I_i}$ then I_i is not allowed to further delegate otherwise the user can delegate and its entry is recorded in TL. From TL, TM can trace that I_i is the original user or delegatee. If $I_k = I_{CA}$ then I_i is the original user.

6.2 Feature analysis

This section gives a comparative analysis of basic delegation in Bethencourt et al. (2007), flexible further delegation in Porwal and Mittal (2019a), traceable delegation Chen et al. (2020) and the proposed scheme based on features

Table 4 Notations required for theoretical analysis

Notations	Description
$\mathbb{G}_0, \mathbb{G}_1$	Bilinear group of prime order p
Z_p	Set of Integer modulo p
S	Set of attributes of a data user
n	Size of S
S_{CT}	Set of attributes associated with ciphertext CT
S_{TN}	Set of transport nodes
O_e	Bilinear pairing operation e
L_X	Bit-Length of the element in X
$\|X\|$	Count of elements in X
$TN_CT(x)$	A threshold gate set of transport node (x)'s children in the access tree
j	Number of nodes in $TN_CT(x)$ of each transport node (x)
$\{Child_1, \ldots, Child_j\}$	Elements of set $TN_CT(x)$
k	Number of different access levels
$\{S_1, \ldots, S_k\}$	Sets of internal nodes associated with different access levels of content
$\{S_{CT_1}, \ldots, S_{CT_k}\}$	Sets of attributes associated with different access mode of ciphertext

required for fully flexible delegation. Table 3 compares the properties of different schemes. Basic delegation given in Bethencourt et al. (2007) supports anonymity as the user can not be identified at decryption. The scheme also supports specific attribute delegation which means the delegatee can access only those contents of the delegator whose access policy is satisfied with the delegated key of specific attributes. Flexible further delegation, given in Porwal and Mittal (2019a) has the property of partial attribute delegation, limited further delegation but the scheme does not support specific content delegation and specific rights delegation. Traceable delegation (Chen et al. 2020) scheme provides a solution to trace the traitor who has illegally delegated his/her key. The proposed scheme is able to trace the unauthorized attempts of decryption as no record would be found in TL for the provided decryption key. An important contribution made through the proposed scheme is specific content delegation, that is, the delegatee will not be able to access all the content whose access policy is satisfied with attributes of the delegated key.

6.3 Theoretical cost analysis

The notations required to define the theoretical analysis are given in Table 4.

Table 5 shows the comparisons of storage and computational overhead of basic delegation (Bethencourt et al. 2007), flexible further delegation (Porwal and Mittal 2019a), traceable delegation (Chen et al. 2020) and the proposed scheme. The computational overhead of the access tree structure and computation of hash has not been considered. It is assumed that there are $k(k = 3)$ fixed hierarchical rights of content. Hence each content is encrypted with k different access keys $k = [1 \ldots 3]$. From Eqs. (18) and (26), it is analyzed that the computational overhead of encryption and decryption operations of the proposed scheme depends on number of children of any transport node (x). Again, according to Eq. (19), the storage overhead of ciphertext CT of the proposed scheme depends on children of a transport node (x). In this analysis, it is assumed that $TN_CT(x)$ of each transport node (x) have j nodes $\{Child_1, \ldots, Child_j\}$. To share content in three different modes (read/write/update) will require three encryption/decryption operations according to three different access structures in Bethencourt et al. (2007), Porwal and Mittal (2019a) and Chen et al. (2020) while in the proposed scheme data owner needs to perform single encryption and data users need to perform single decryption for all three-access structure. As illustrated in Table 5, analysis says that the computational overhead of encryption and decryption operation increases linearly with respect to the number of access keys k in the proposed scheme with fixed j. Hence computational overhead of the encryption and decryption operations of k access keys in the proposed

Table 5 Comparisons of computational and storage overhead of basic delegation (Bethencourt et al. 2007), Flexible Further Delegation (Porwal and Mittal 2019a), Traceable Delegation (Chen et al. 2020) and the proposed scheme

Parameter	Basic delegation (Bethencourt et al. 2007)	Flexible further delegation (Porwal and Mittal 2019a)	Traceable delegation (Chen et al. 2020)	Proposed scheme
Computational overhead of Encryption	$\left[2\left(\lvert S_{CT_1}\rvert + \cdots + \lvert S_{CT_k}\rvert\right) + k\right]\mathbb{G}_0 + 2k\mathbb{G}_1$	$\left[2\left(\lvert S_{CT_1}\rvert + \cdots + \lvert S_{CT_k}\rvert\right) + k\right]\mathbb{G}_0 + 2k\mathbb{G}_1$	$\left[2\left(\lvert S_{CT_1}\rvert + \cdots + \lvert S_{CT_k}\rvert\right) + k\right]\mathbb{G}_0 + 2k\mathbb{G}_1$	$\left(2\lvert S_{CT_1}\rvert + k\right)\mathbb{G}_0 + (2j\lvert S_{TN}\rvert + 2k)\mathbb{G}_1$
Computational overhead of Decryption	$k(2\lvert S\rvert + 1)O_e + [2(\lvert S_1\rvert + \cdots + \lvert S_k\rvert) + 2k]\mathbb{G}_1$	$k(2\lvert S\rvert + 1)O_e + [2(\lvert S_1\rvert + \cdots + \lvert S_k\rvert) + 2k]\mathbb{G}_1$	$k(2\lvert S\rvert + 1)O_e + [2(\lvert S_1\rvert + \cdots + \lvert S_k\rvert) + 2k]\mathbb{G}_1$	$(\lvert S\rvert + 1)O_e + [2\lvert S_1\rvert + (j\lvert S_{TN}\rvert + 2k)]\mathbb{G}_1$
Storage overhead of PP	$3L_{\mathbb{G}_0} + L_{\mathbb{G}_1}$	$(n + 1)L_{\mathbb{G}_0} + L_{\mathbb{G}_1}$	$(n + l)L_{\mathbb{G}_0} + L_{\mathbb{G}_1}$	$(n + l + 3)L_{\mathbb{G}_0} + L_{\mathbb{G}_1}$
Storage overhead of MSK	$L_{Z_p} + L_{\mathbb{G}_0}$	$(n + 1)L_{Z_p}$	$L_{\mathbb{G}_0} + (n + l + 1)L_{Z_p}$	$L_{\mathbb{G}_0} + (n + l + 1)L_{Z_p}$
Storage overhead of SK	$(2\lvert S\rvert + 1)L_{\mathbb{G}_0}$	$(2\lvert S\rvert + 1)L_{\mathbb{G}_0}$	$\lvert S'\rvert L_{\mathbb{G}_0}$	$(2\lvert S'\rvert + 1)L_{\mathbb{G}_0}$
Storage overhead of CT	$\left[2\left(\lvert S_{CT_1}\rvert + \cdots + \lvert S_{CT_k}\rvert\right) + k\right]L_{\mathbb{G}_0} + kL_{\mathbb{G}_1}$	$\left[2\left(\lvert S_{CT_1}\rvert + \cdots + \lvert S_{CT_k}\rvert\right) + k\right]L_{\mathbb{G}_0} + kL_{\mathbb{G}_1}$	$\left[2\left(\lvert S_{CT_1}\rvert + \cdots + \lvert S_{CT_k}\rvert\right) + k\right]L_{\mathbb{G}_0} + kL_{\mathbb{G}_1}$	$\left(2\lvert S_{CT_1}\rvert + k\right)L_{\mathbb{G}_0} + (2j\lvert S_{TN}\rvert + 2k)L_{\mathbb{G}_1}$

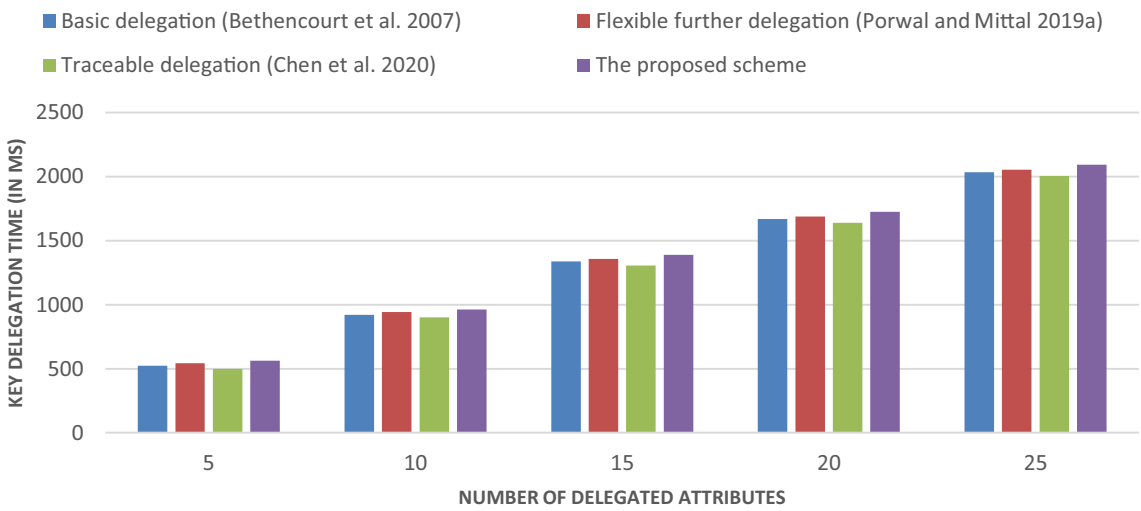


Fig. 3 Key delegation time with respect to varying number of delegated attributes

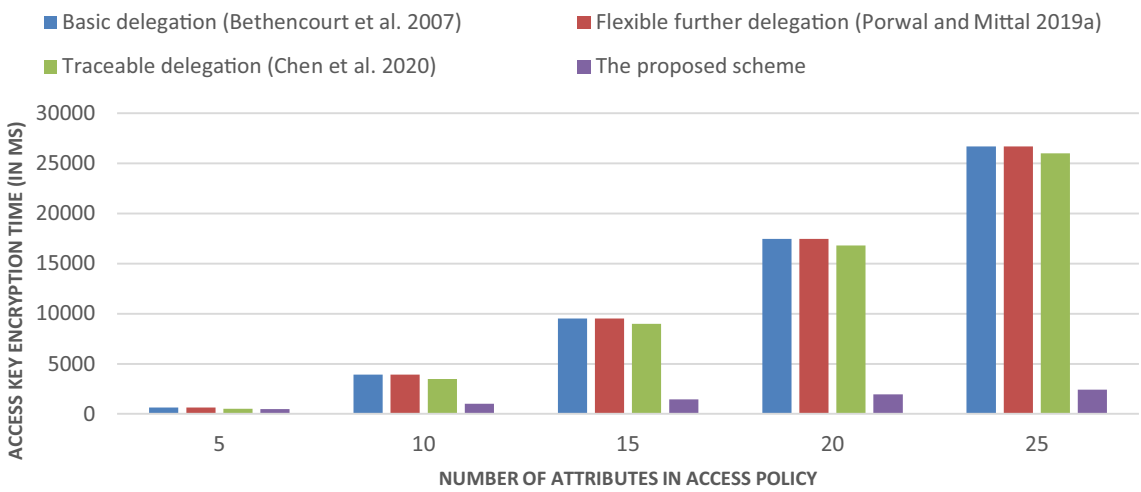


Fig. 4 Encryption time for largest possible access key with respect to varying number of attributes

scheme are relatively smaller than Bethencourt et al. (2007), Porwal and Mittal (2019a) and Chen et al. (2020) due to the reduction of a few steps. Moreover, in all Bethencourt et al. (2007), Porwal and Mittal (2019a), Chen et al. (2020) the and the proposed scheme, the storage overhead of PP, MSK is fixed and storage overhead of SK depends on the number of attributes of the user.

6.4 Performance analysis

This subsection presents a comparative performance analysis of the proposed scheme and the basic delegation (Bethencourt et al. 2007), flexible further delegation (Porwal and Mittal 2019a) and traceable delegation (Chen et al. 2020) on the basis of computational cost. The impact of the computational overhead of the proposed scheme and the basic delegation (Bethencourt et al. 2007), flexible further delegation (Porwal and Mittal 2019a) and traceable delegation (Chen et al. 2020) has been evaluated. The computational cost of encryption, decryption and key delegation cryptographic operations has been analyzed.

The proposed scheme has been implemented in Java based on the Java Pairing-Based Cryptography library (JPBC) (De Caro and Iovino 2011). The implementation is based on the 160-bit elliptic curve over the finite field of 512-bit. To compare the implementation results of key delegation, encryption and decryption functions of basic delegation (Bethencourt et al. 2007), flexible further delegation (Porwal and Mittal 2019a) and traceable delegation (Chen et al. 2020) have been done. The experiments are done on the Java platform on an Intel(R) Core (TM) i3-6006U CPU machine @ 2.00 GHz and 8.00 GB RAM

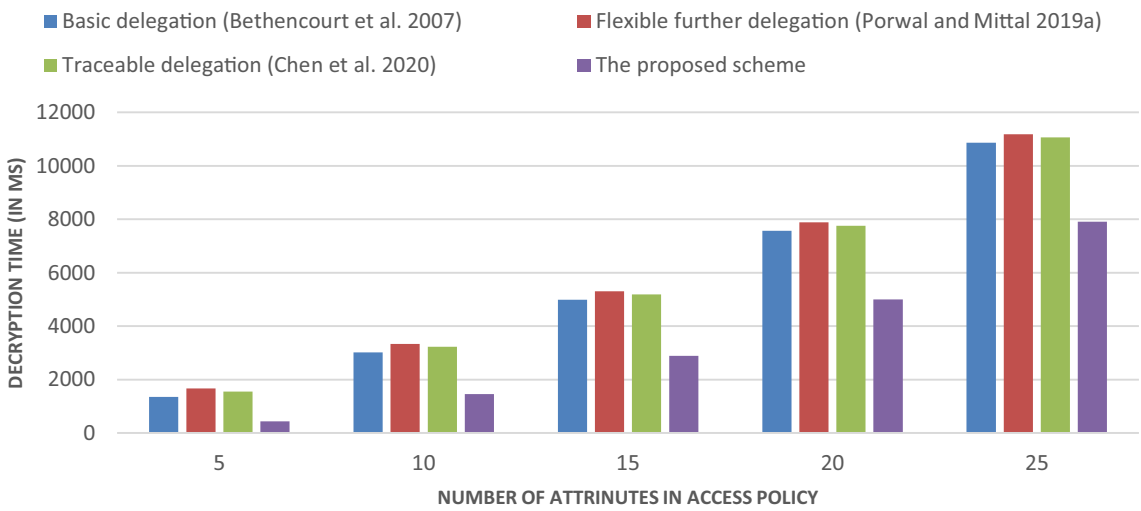


Fig. 5 Decryption time for largest possible access key with respect to varying number of attributes

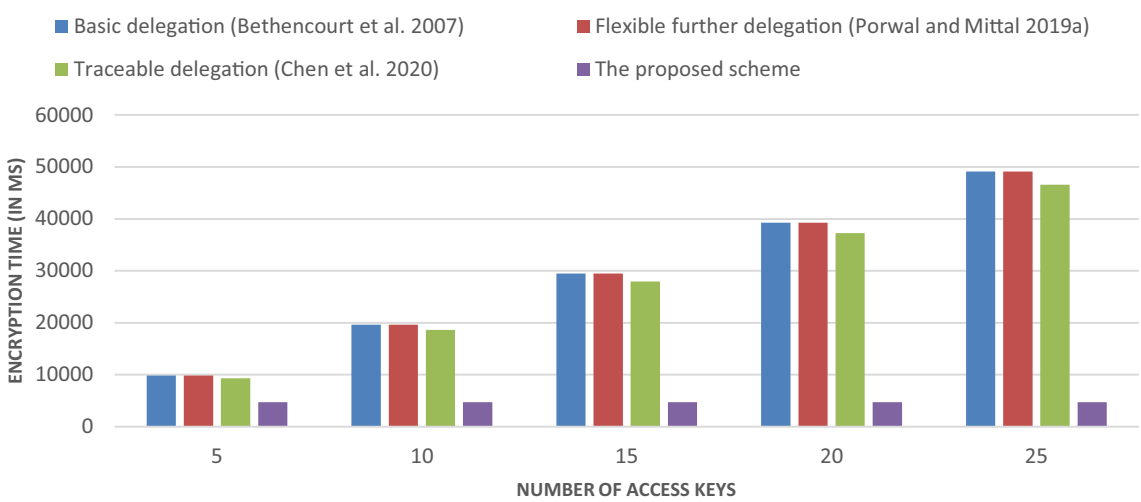


Fig. 6 Encryption time with respect to varying number of access keys (fixed number of attributes in access policy N = 26)

and the operating system is Windows 10 and the data set is text file. In all experiments, results of the mean of ten runs have been presented.

The cost of the key delegation is shown in Fig. 3. The proposed key delegation scheme requires one update in TL and one write in CL and the cost of these updates and writes is 60 ms only. Hence the proposed scheme provides a fully flexible content delegation mechanism with fine grained access control with improved efficiency of encryption and decryption.

To encrypt k different access keys $\{key_1, \ldots, key_k\}$ associated with a content$m$, the proposed scheme designs access policy as $\{attr_1 attr_2 2of2 attr_3 2of2 \ldots attr_k 2of2 attr_{k+1} 2of2\}$.

On the other hand, in basic delegation (Bethencourt et al. 2007), flexible further delegation (Porwal and Mittal 2019a) and traceable delegation (Chen et al. 2020) k different access policies are designed as $\{attr_1 attr_2 2of2\}$, $\{attr_1 attr_2 2of2 attr_3 2of2\}$, $\{attr_1 attr_2 2of2 attr_3 2of2 \ldots attr_k 2of2 attr_{k+1} 2of2\} \ldots$.

As shown in Figs. 4 and 5, the encryption and decryption cost of k different access keys $\{key_1, \ldots, key_k\}$ associated with any content is reduced by the proposed scheme because it encrypts and decrypts multiple access keys using a single access structure. Sharing content in three different modes (read/write/update) will require three encryption/decryption operations in Bethencourt et al. (2007), Porwal and Mittal (2019a) and Chen et al. (2020) while in the proposed scheme this can be done in single encryption and decryption operation. For performance analysis, access policy is designed using AND gate ($2of2thresholdgate$) so that all attributes

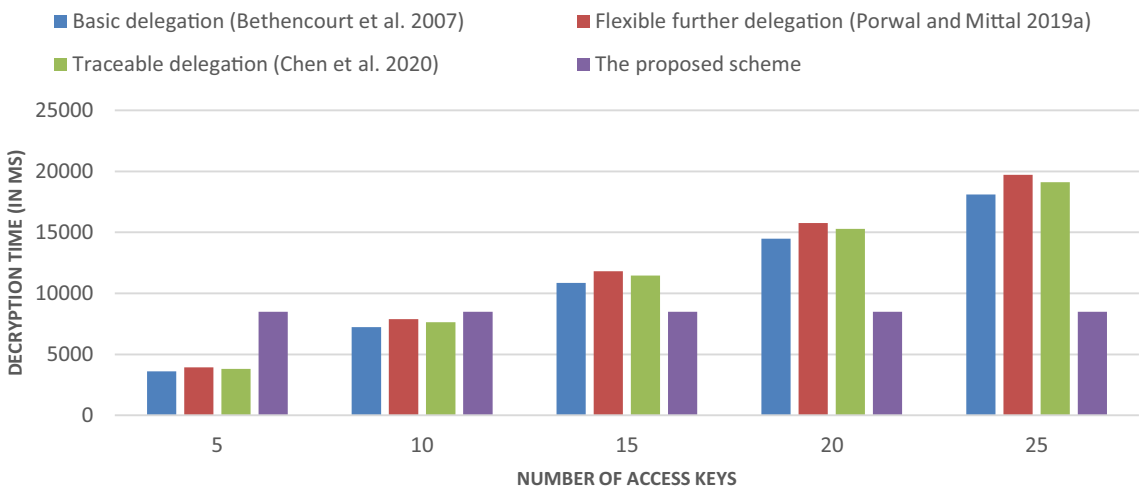


Fig. 7 Decryption time with respect to varying number of access keys (fixed number of attributes in access policy N = 26)

of access policy attached with ciphertext are computed at decryption. The encryption cost of 25 access keys using basic delegation (Bethencourt et al. 2007), flexible further delegation (Porwal and Mittal 2019a) traceable delegation (Chen et al. 2020) and the proposed scheme are 26708 ms, 26708 ms, 26708 ms, and 2410 ms respectively. Hence to encrypt 25 access keys using the proposed scheme will require only 9% cost of encryption using Bethencourt et al. (2007), Porwal and Mittal (2019a) and Chen et al. (2020). The decryption cost of 25 access keys using basic delegation (Bethencourt et al. 2007), flexible further delegation (Porwal and Mittal 2019a), traceable delegation (Chen et al. 2020) and proposed scheme are 10861 ms, 11078 ms, 10871 ms and 7911 ms respectively. Hence decrypting 25 access keys using the proposed scheme will require almost 72% cost of decryption using Bethencourt et al. (2007), Porwal and Mittal (2019a) and Chen et al. (2020).

The proposed scheme has been evaluated for its performance on a fixed number of attributes. In Figs. 6 and 7, it can be seen that the proposed scheme reduces the cost of encryption and decryption significantly with a fixed number of attributes ($N = 26$). For example, the encryption cost of 25 access keys using basic delegation (Bethencourt et al. 2007), flexible further delegation (Porwal and Mittal 2019a) and traceable delegation (Chen et al. 2020) and the proposed scheme are 49100 ms, 49100 ms, 49100 ms and 4731 ms. Hence to encrypt 25 access keys using the proposed scheme requires only 9.6% cost of encryption using Bethencourt et al. (2007), Porwal and Mittal (2019a) and Chen et al. (2020). The decryption cost of 25 access keys using basic delegation (Bethencourt et al. 2007), flexible further delegation (Porwal and Mittal 2019a), traceable delegation (Chen et al. 2020) and the proposed scheme are 18100 ms, 18140 ms, 18120 ms and 8482 ms respectively. Hence to decrypt 25 access keys using the proposed scheme will require almost 46% cost of decryption using Bethencourt et al. (2007), Porwal and Mittal (2019a) and Chen et al. (2020).

7 Conclusion

In this research work, a fully flexible key delegation mechanism with fine grained access control has been proposed. The scheme efficiently shares different access keys associated with the content. The scheme provides specific content delegation, specific access rights delegation, and limited access of delegated contents. It also provides identity-based delegatee/user revocation with traceability. Performance evaluation shows that the proposed scheme is more efficient than existing ones and has lesser cost of encryption and decryption.

References

Sahai A, Waters B (2005) Fuzzy identity-based encryption. In: Annual international conference on the theory and applications of cryptographic techniques (pp 457–473). Springer, Berlin, Heidelberg

Porwal S, Mittal S (2017) Implementation of Ciphertext Policy-Attribute Based Encryption (CP-ABE) for fine grained access control of university data. In: 2017 tenth international conference on contemporary computing (IC3), IEEE, pp 1–7

Bethencourt J, Sahai A, Waters B (2007) Ciphertext-policy attribute-based encryption. In: 2007 IEEE symposium on security and privacy (SP'07), IEEE, pp 321–334

Ibraimi L, Petkovic M, Nikova S, Hartel P, Jonker W (2009) Ciphertext-policy attribute-based threshold decryption with flexible delegation and revocation of user attributes. IEEE Trans Image Process

Jiang Y, Susilo W, Mu Y, Guo F (2016) Ciphertext-policy attribute-based encryption with key delegation abuse resistance. In:

Australasian conference on information security and privacy. Springer, Cham, pp 477–494
Jiang Y, Susilo W, Mu Y, Guo F (2018) Ciphertext-policy attribute-based encryption against key-delegation abuse in fog computing. Futur Gener Comput Syst 78:720–729
Porwal S, Mittal S (2019a) A secure key delegation mechanism for fog networking. In: 2019a twelfth international conference on contemporary computing (IC3), IEEE, pp 1–7
Porwal S, Mittal S (2019b) A flexible secure key delegation mechanism for CP-ABE with hidden access structure. In: 2019b 11th international conference on information technology and electrical engineering (ICITEE), IEEE, pp 1–6
Xu S, Yang G, Mu Y (2019) Revocable attribute-based encryption with decryption key exposure resistance and ciphertext delegation. Inf Sci 479:116–134
Ma H, Wang Z, Guan Z (2019) Efficient ciphertext-policy attribute-based online/offline encryption with user revocation. Secur Commun Netw 2019:5
Liu Z, Xu J, Liu Y, Wang B (2019) Updatable ciphertext-policy attribute-based encryption scheme with traceability and revocability. IEEE Access 7:66832–66844
Lian H, Wang Q, Wang G (2020) Large universe ciphertext-policy attribute-based encryption with attribute level user revocation in cloud storage. Int Arab J Inf Technol 17(1):107–117
Cao Z, Markowitch O (2020) Comment on "circuit ciphertext-policy attribute-based hybrid encryption with verifiable delegation in cloud computing." IEEE Trans Parallel Distrib Syst 32(2):392–393
Ali M, Mohajeri J, Sadeghi MR, Liu X (2020a) A fully distributed hierarchical attribute-based encryption scheme. Theor Comput Sci 815:25–46
Chen X, Liu Y, Chao HC, Li Y (2020) Ciphertext-policy hierarchical attribute-based encryption against key-delegation abuse for IoT-connected healthcare system. IEEE Access 8:86630–86650
Cui H, Deng RH, Qin B, Weng J (2020) Key regeneration-free ciphertext-policy attribute-based encryption and its application. Inf Sci 517:217–229
Ali M, Sadeghi MR, Liu X (2020b) Lightweight revocable hierarchical attribute-based encryption for internet of things. IEEE Access 8:23951–23964
Zhang W, Zhang Z, Xiong H, Qin Z (2021) PHAS-HEKR-CP-ABE: partially policy-hidden CP-ABE with highly efficient key revocation in cloud data sharing system. J Ambient Intell Human Comput 13:1–15
Huang Q, Wang L, Yang Y (2018) DECENT: Secure and fine-grained data access control with policy updating for constrained IoT devices. World Wide Web 21(1):151–167
Ahuja R, Mohanty SK (2017) A scalable attribute-based access control scheme with flexible delegation cum sharing of access privileges for cloud storage. IEEE Trans Cloud Comput 8(1):32–44
Wang Q, Peng L, Xiong H, Sun J, Qin Z (2017) Ciphertext-policy attribute based encryption with delegated equality test in cloud computing. IEEE Access 6:760–771
Pussewalage HSG, Oleshchuk VA (2017) Attribute based access control scheme with controlled access delegation for collaborative E-health environments. J Inf Secur Appl 37:50–64
Xu J, Wen Q, Li W, Jin Z (2015) Circuit ciphertext-policy attribute based hybrid encryption with verifiable delegation in cloud computing. IEEE Trans Parallel Distrib Syst 27(1):119–129
Premkamal PK, Pasupuleti SK, Alphonse PJA (2019) A new verifiable outsourced ciphertext-policy attribute based encryption for big data privacy and access control in cloud. J Ambient Intell Humaniz Comput 10(7):2693–2707
Premkamal PK, Pasupuleti SK, Alphonse PJA (2021) Dynamic traceable CP-ABE with revocation for outsourced big data in cloud storage. Int J Commun Syst 34(2):e4351
Liang K, Fang L, Susilo W, Wong DS (2013) A ciphertext-policy attribute-based proxy re-encryption with chosen-ciphertext security. In: 2013 5th international conference on intelligent networking and collaborative systems, IEEE, pp 552–559
Zhang Y, Li J, Chen X, Li H (2016) Anonymous attribute-based proxy re-encryption for access control in cloud computing. Secur Commun Netw 9(14):2397–2411
Canetti R, Krawczyk H, Nielsen JB (2003) Relaxing chosen-ciphertext security. In: Annual international cryptology conference. Springer, Berlin, Heidelberg, pp 565–582
De Caro A, Iovino V (2011) jPBC: Java pairing based cryptography. In: 2011 IEEE symposium on computers and communications (ISCC). IEEE, pp 850–855

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor holds exclusive rights to this article under a publishing agreement with theauthor(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solelygoverned by the terms of such publishing agreement and applicable law.